

POLITECNICO
MILANO 1863

# Bioinformatics Algorithms

## Read mapping (part 2)

Rosario M. Piro
(based on slides by Peter N. Robinson)

Dipartimento di Elettronica, Informazione e Bioingegneria (DEIB)
Politecnico di Milano

# Burrows Wheeler Transform (BWT)

The BWT applies a reversible transformation to a block of input text. The transformation does not itself compress the data, but reorders it to make it easy to compress with simple algorithms such as move-to-front coding

Original publication: Burrows & Wheeler, 1994

- The significance of the BWT for most of the world is its use for data compression
  - Leads to a block-sorted data structure (runs of repeated characters)
  - E.g., basis for the bzip2 compression algorithm

- Basis for many of the read mapping algorithms in common use today
  - The obtained block-sorted data structure is also well suited to searching short strings in a larger text (here: reads in a genome)
  - The FM index uses the BWT, the suffix array and additional data structures to enable search with time linear in the length of the search string (length of the read!) (see Ferragina & Manzini, 2000)

# Burrows Wheeler Transform (BWT)

**First step**: form all **rotations** of the input text $T$
Note that as with the suffix array and suffix tree, we append a **termination character** $ to the end of the text

```
T="abracadabra$“

 0: abracadabra$
 1: bracadabra$a
 2: racadabra$ab
 3: acadabra$abr
 4: cadabra$abra
 5: adabra$abrac
 6: dabra$abraca
 7: abra$abracad
 8: bra$abracada
 9: ra$abracadab
10: a$abracadabr
11: $abracadabra
```

# Burrows Wheeler Transform (BWT)

**Second step**: Sort the rotated strings lexicographically
The result is the **Burrows Wheeler matrix**

```
 0: abracadabra$          0: $abracadabra
 1: bracadabra$a          1: a$abracadabr
 2: racadabra$ab          2: abra$abracad
 3: acadabra$abr          3: abracadabra$
 4: cadabra$abra          4: acadabra$abr
 5: adabra$abrac  sort    5: adabra$abrac
 6: dabra$abraca  ---->   6: bra$abracada
 7: abra$abracad          7: bracadabra$a
 8: bra$abracada          8: cadabra$abra
 9: ra$abracadab          9: dabra$abraca
10: a$abracadabr         10: ra$abracadab
11: $abracadabra         11: racadabra$ab
```

Recall that, lexicographically the termination character $ comes before every other character!

# Burrows Wheeler Transform (BWT)

**Third step**: The **Burrows Wheeler Transform** is simply the last column of the **Burrows Wheeler matrix**

```
$abracadabra
a$abracadabr
abra$abracad
abracadabra$
acadabra$abr
adabra$abrac
bra$abracada
bracadabra$a
cadabra$abra
dabra$abraca
ra$abracadab
racadabra$ab
```

Note: since we rotated all characters, the last column contains the same set of characters as $T$

# Burrows Wheeler Transform (BWT)

We will denote the Burrows Wheeler transform of an input string $T$ as

$$\mathrm{BWT}(T)$$

- Thus, $\mathrm{BWT}(T)$="ard$rcaaaabb" for $T$="abracadabra$"
- It is relatively easy to implement a naive version of the BWT
  1. Create all rotations of $T$
  2. Sort the rotations lexicographically
  3. Concatenate the last character of each rotation to form BWT($T$)
- The BWT tends to contain lots of "runs" of identical characters, which is a good feature to have for compression algorithms such as **run-length encoding**
  - The examples we use here are too short to appreciate this, but consider the following excerpt of BWT(*Macbeth*, Act 1, Scene 1):

    ```
    ...uoaoiiiiiiiiiiiiiiiaaaaaiiiiiuiiiiiiiiiiiiiiiaAAiiiiiiioieei...
    ```

    A simple run-length encoding might be

    ```
    ...uoaoi{15}a{5}i{5}ui{17}aA{2}i{7}oie{2}i...
    ```

# BWT and Suffix array

**BW matrix**

```
$abracadabra
a$abracadabr
abra$abracad
abracadabra$
acadabra$abr
adabra$abrac
bra$abracada
bracadabra$a
cadabra$abra
dabra$abraca
ra$abracadab
racadabra$ab
```

**Suffix array with corresponding suffixes**

```
[11]  $
[10]  a$
[7]   abra$
[0]   abracadabra$
[3]   acadabra$
[5]   adabra$
[8]   bra$
[1]   bracadabra$
[4]   cadabra$
[6]   dabra$
[9]   ra$
[2]   racadabra$
```

- The Burrows Wheeler matrix is (nearly) the same as the suffixes referred to by the suffix array of the same string

# BWT and Suffix array

We can now write an algorithm to create $BWT(T)$ from the suffix array $SA(T)$ of $T$ by noting that position $i$ of the BWT corresponds to the character that, in the original string, is just to the left of the $i$-th suffix in the SA

Explanation: this is exactly the character that has been "rotated" to the back of the BW matrix!

**BW matrix** **Suffix array with corresponding suffixes**

Consider the third sorted rotation in the BWM and the third suffix in the suffix array for $T$=abracadabra$

abra$abracad [7] abra$

The character just to the left of the suffix is the $i^{th}$ character of BWT($T$)

$T$=abracad**a**bra$

# BWT and Suffix array

**BW matrix** **Suffix array with corresponding suffixes**

Consider the third sorted rotation in the BWM and the third suffix in the suffix array for *T*=abracadabra$

abra$abracad [7] abra$

The character just to the left of the suffix is the $i^{th}$ character of BWT(*T*)

*T*=abracadabra$

- We can now construct the BWT as follows:
  (where $SA[i]$ is the position/offset in $T$ of the $i$-th sorted suffix)

$$\mathrm{BWT}(T)[i] = \begin{cases} T[SA[i]-1] & \text{if} \quad SA[i] > 0 \\ \$ & \text{if} \quad SA[i] = 0 \end{cases} \tag{1}$$

  To see the reason for the second case, consider that the first suffix of the suffix array is always $

# BWT and Suffix array

```
T=abracadabra$
  012345678901
```

$$\mathrm{BWT}(T)[i] = \begin{cases} T[SA[i]-1] & \text{if} \quad SA[i] > 0 \\ \$ & \text{if} \quad SA[i] = 0 \end{cases}$$

**BW matrix**

```
$abracadabra
a$abracadabr
abra$abracad
abracadabra$
acadabra$abr
adabra$abrac
bra$abracada
bracadabra$a
cadabra$abra
dabra$abraca
ra$abracadab
racadabra$ab
```

**Suffix array with corresponding suffixes**

```
[11] $
[10] a$
[7]  abra$
[0]  abracadabra$
[3]  acadabra$
[5]  adabra$
[8]  bra$
[1]  bracadabra$
[4]  cadabra$
[6]  dabra$
[9]  ra$
[2]  racadabra$
```

**(Work through example)**

# Constructing a BWT from a Suffix Array

The naive algorithm is pretty simple to implement

**Algorithm 1** bwtFromSuffixArray($T$)

```
1:  sa = constructSuffixArray(T$)
2:  L = length(sa)
3:  bwt = new string[L]
4:  for i=0 to i=L-1 do
5:      if sa[i] = 0 then
6:          bwt[i] = $
7:      else
8:          bwt[i] = T[sa[i] − 1]
9:      end if
10: end for
11: return bwt
```

# Reversing the BWT

If we want to get the original string back, we need to

1. Reverse the compression procedure (in case we compressed the BWT)
2. Get the original string back from the BWT

So, how *do* we reverse the Burrows Wheeler transformation?

The reversibility of the BWT depends on the

## LF Mapping property

For any character, the $T$-ranking of characters in the first column (**F**) of the BW matrix is the same as order of characters in the last column (**L**)

# Reversing the BWT

So, what is the T-ranking?

$$\mathtt{a}_0\mathtt{b}_0\mathtt{r}_0\mathtt{a}_1\mathtt{c}_0\mathtt{a}_2\mathtt{d}_0\mathtt{a}_3\mathtt{b}_1\mathtt{r}_1\mathtt{a}_4\$$$

- The $T$-ranking of the character at any given position is the number of times that an identical character has preceeded it in $T$
- The $T$-ranking of \$ is always zero (no previous \$!) and is omitted here
- Important: the ranks are shown here just to help understand the LF mapping property; the $T$-ranks are not stored explicitly!

# Reversing the BWT

$$
\begin{array}{l}
\$_0 a_0 b_0 r_0 a_1 c_0 a_2 d_0 a_3 b_1 r_1 a_4 \\
a_4 \$_0 a_0 b_0 r_0 a_1 c_0 a_2 d_0 a_3 b_1 r_1 \\
a_3 b_1 r_1 a_4 \$_0 a_0 b_0 r_0 a_1 c_0 a_2 d_0 \\
a_0 b_0 r_0 a_1 c_0 a_2 d_0 a_3 b_1 r_1 a_4 \$_0 \\
a_1 c_0 a_2 d_0 a_3 b_1 r_1 a_4 \$_0 a_0 b_0 r_0 \\
a_2 d_0 a_3 b_1 r_1 a_4 \$_0 a_0 b_0 r_0 a_1 c_0 \\
b_1 r_1 a_4 \$_0 a_0 b_0 r_0 a_1 c_0 a_2 d_0 a_3 \\
b_0 r_0 a_1 c_0 a_2 d_0 a_3 b_1 r_1 a_4 \$_0 a_0 \\
c_0 a_2 d_0 a_3 b_1 r_1 a_4 \$_0 a_0 b_0 r_0 a_1 \\
d_0 a_3 b_1 r_1 a_4 \$_0 a_0 b_0 r_0 a_1 c_0 a_2 \\
r_1 a_4 \$_0 a_0 b_0 r_0 a_1 c_0 a_2 d_0 a_3 b_1 \\
r_0 a_1 c_0 a_2 d_0 a_3 b_1 r_1 a_4 \$_0 a_0 b_0
\end{array}
$$

- Here is the Burrows Wheeler matrix with the $T$-ranks of all the characters

# Reversing the BWT

$$
\begin{array}{l}
\$_0 a_0 b_0 r_0 a_1 c_0 a_2 d_0 a_3 b_1 r_1 a_4 \\
a_4 \$_0 a_0 b_0 r_0 a_1 c_0 a_2 d_0 a_3 b_1 r_1 \\
a_3 b_1 r_1 a_4 \$_0 a_0 b_0 r_0 a_1 c_0 a_2 d_0 \\
a_0 b_0 r_0 a_1 c_0 a_2 d_0 a_3 b_1 r_1 a_4 \$_0 \\
a_1 c_0 a_2 d_0 a_3 b_1 r_1 a_4 \$_0 a_0 b_0 r_0 \\
a_2 d_0 a_3 b_1 r_1 a_4 \$_0 a_0 b_0 r_0 a_1 c_0 \\
b_1 r_1 a_4 \$_0 a_0 b_0 r_0 a_1 c_0 a_2 d_0 a_3 \\
b_0 r_0 a_1 c_0 a_2 d_0 a_3 b_1 r_1 a_4 \$_0 a_0 \\
c_0 a_2 d_0 a_3 b_1 r_1 a_4 \$_0 a_0 b_0 r_0 a_1 \\
d_0 a_3 b_1 r_1 a_4 \$_0 a_0 b_0 r_0 a_1 c_0 a_2 \\
r_1 a_4 \$_0 a_0 b_0 r_0 a_1 c_0 a_2 d_0 a_3 b_1 \\
r_0 a_1 c_0 a_2 d_0 a_3 b_1 r_1 a_4 \$_0 a_0 b_0
\end{array}
$$

- What do you notice about the $T$-ranks of the “a” characters?

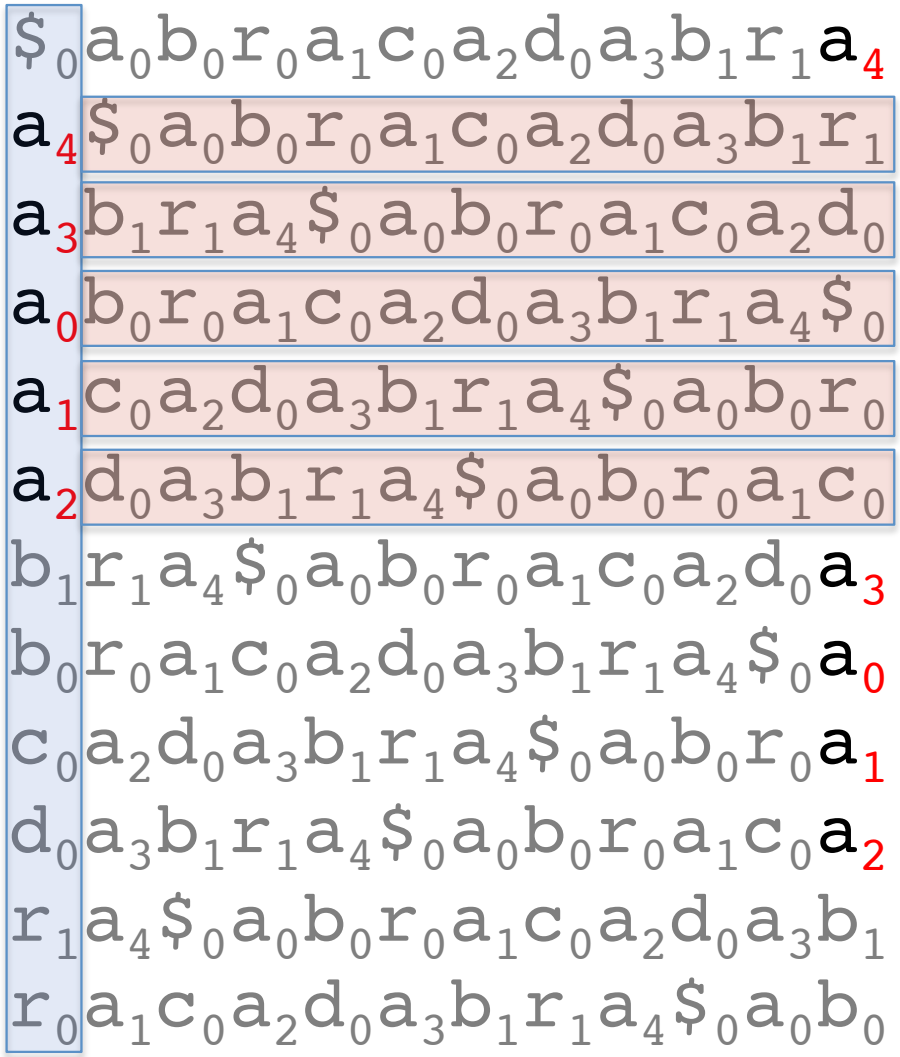
# Reversing the BWT

- The “a”s have the same relative order in the **F** and the **L** columns
- A similar observation pertains to the other characters
  - E.g., first “$r_1$”, then “$r_0$”
- Can you explain this?

  Idea: let’s have a look of the lexicographical sorting!

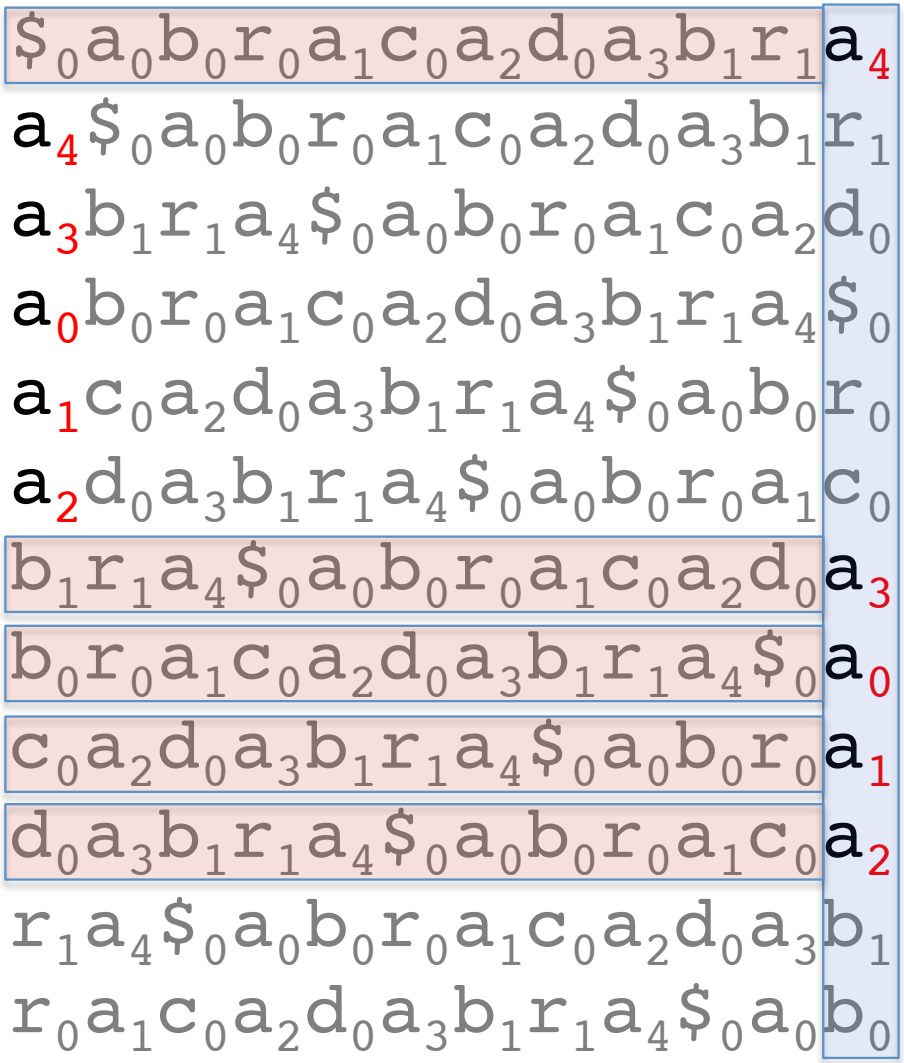
$$
\begin{array}{l}
\$_0a_0b_0r_0a_1c_0a_2d_0a_3b_1r_1a_4 \\
a_4\$_0a_0b_0r_0a_1c_0a_2d_0a_3b_1r_1 \\
a_3b_1r_1a_4\$_0a_0b_0r_0a_1c_0a_2d_0 \\
a_0b_0r_0a_1c_0a_2d_0a_3b_1r_1a_4\$_0 \\
a_1c_0a_2d_0a_3b_1r_1a_4\$_0a_0b_0r_0 \\
a_2d_0a_3b_1r_1a_4\$_0a_0b_0r_0a_1c_0 \\
b_1r_1a_4\$_0a_0b_0r_0a_1c_0a_2d_0a_3 \\
b_0r_0a_1c_0a_2d_0a_3b_1r_1a_4\$_0a_0 \\
c_0a_2d_0a_3b_1r_1a_4\$_0a_0b_0r_0a_1 \\
d_0a_3b_1r_1a_4\$_0a_0b_0r_0a_1c_0a_2 \\
r_1a_4\$_0a_0b_0r_0a_1c_0a_2d_0a_3b_1 \\
r_0a_1c_0a_2d_0a_3b_1r_1a_4\$_0a_0b_0
\end{array}
$$

# Reversing the BWT



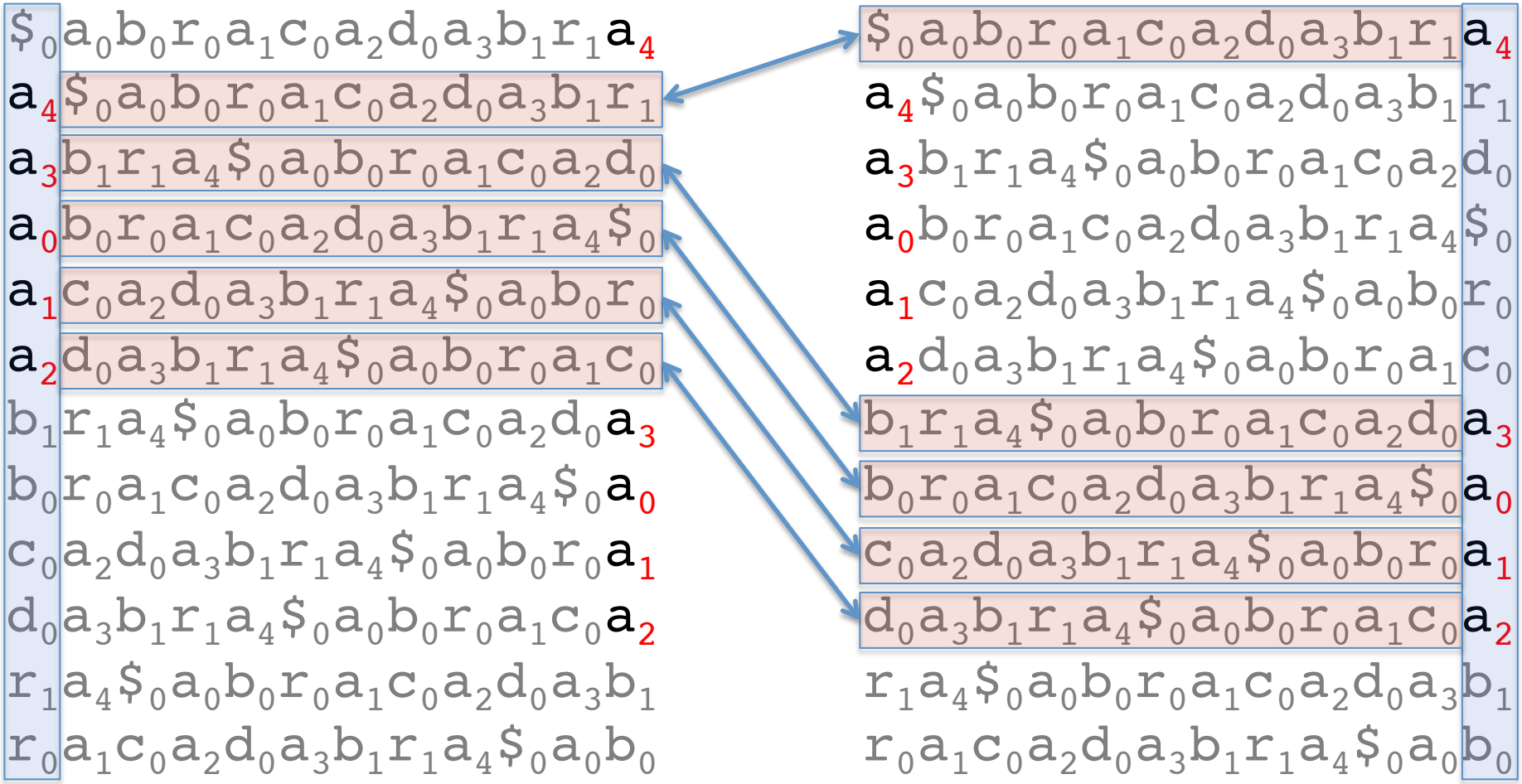

- The relative $T$-ranks of the a characters in column **F** are determined by the lexicographic ranks of the strings **to the right** of the characters

# Reversing the BWT




- The relative $T$-ranks of the a characters in column **L** must reflect the lexicographic ranks of the strings **to the "rotated" right** of the characters

# Reversing the BWT




- These are the same strings (consequence of the rotation!)
- Thus: if the sorting of “a”s in **F** and **L** is determined by exactly the same strings, their sorting ($T$-ranking) must be exactly the same!

# Reversing the BWT

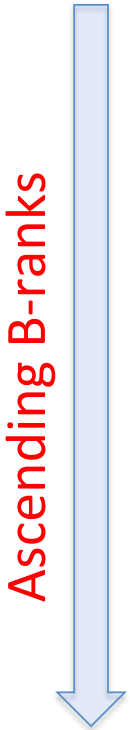
- Do we really need to remember/store the $T$-ranking?
- We introduce another "vertical" ranking
- The **B-ranking** of a character at a specific position is the number of that times the same character has occured in the **F** column "above" the current position
- The B-ranking is thus like a cumulative count of the characters
- Just a re-labeling of identical characters, so also the B-ranking is the same for the **F** and **L** columns!

$\$_0$abracadabra$_0$
a$_0$\$abracadabr$_0$
a$_1$bra\$abracad$_0$
a$_2$bracadabra\$$_0$
a$_3$cadabra\$abr$_1$
a$_4$dabra\$abrac$_0$
b$_0$ra\$abracada$_1$
b$_1$racadabra\$a$_2$
c$_0$adabra\$abra$_3$
d$_0$abra\$abraca$_4$
r$_0$a\$abracadab$_0$
r$_1$acadabra\$ab$_1$

Just the **F** and **L** columns are shown for better legibility

# Reversing the BWT

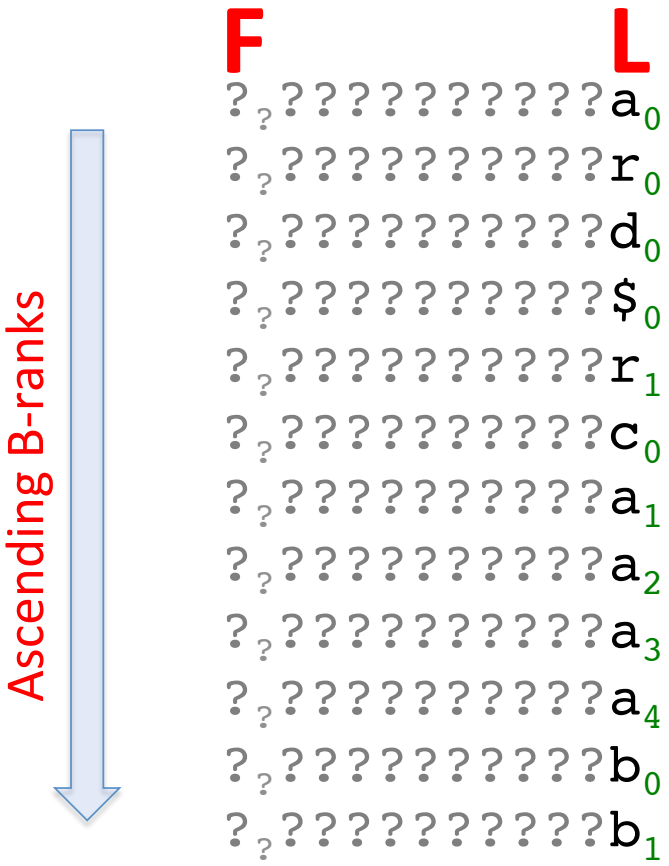
- Column **F** has a simple structure: chunks of identical characters with ascending B-ranks
  - Remember: they are lexicographically sorted!
- Column **L** does not generally have this kind of strict chunk structure, but the B-ranks of any given character also are arranged in ascending order




| F | L |
|---|---|
| $$_0$$ | a$_0$ |
| a$_0$ | b$_0$ |
| a$_1$ | r$_0$ |
| a$_2$ | c$_0$ |
| a$_3$ | d$_0$ |
| a$_4$ | r$_1$ |
| b$_0$ | $$_0$$ |
| b$_1$ | a$_1$ |
| c$_0$ | a$_2$ |
| d$_0$ | a$_3$ |
| r$_0$ | a$_4$ |
| r$_1$ | b$_1$ |

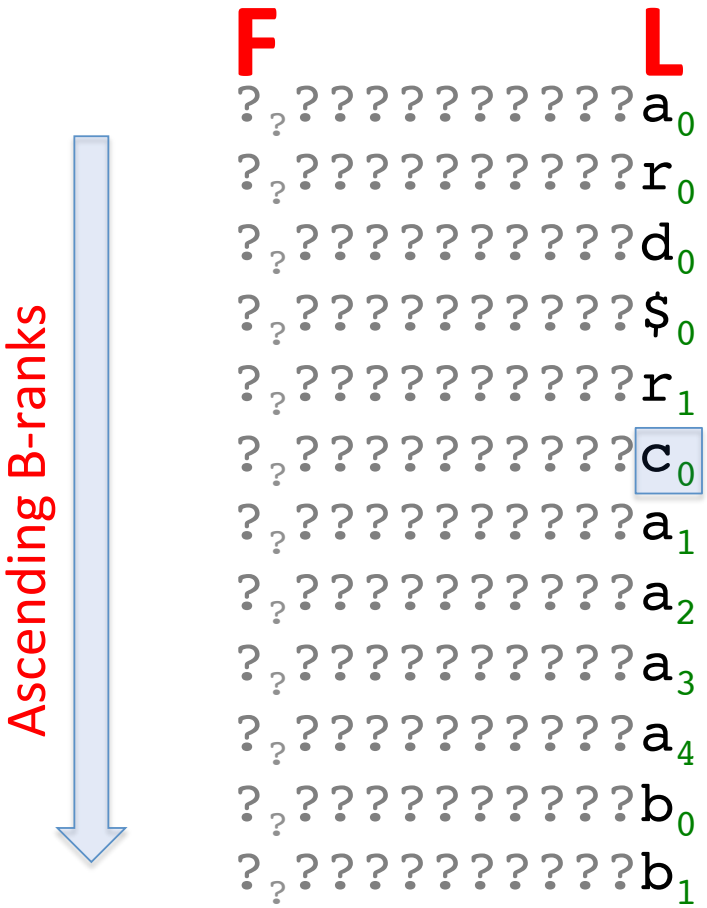
# Reversing the BWT

- Can we now use these observations to reconstruct the original string?
- We will first try to reconstruct the first column of the BWM



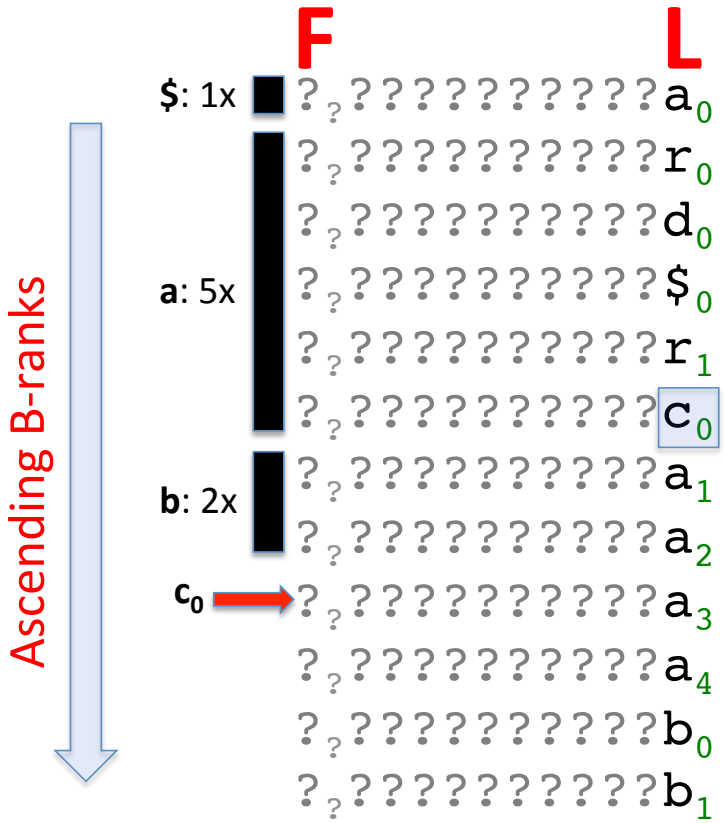
# Reversing the BWT

- Consider $c_0$
- We know that the \$, all the “a”s, all the “b”s, but not any of the “d”s must precede $c_0$ in the first column
  - The number of identical characters in **F** is the same as in **L**, so we can simply count them in the BWT!



# Reversing the BWT

- The index of $c_0$ in column **F** must equal 1+5+2=8
- We will refer to this as the **cumulative index property**
  - This will help us to find a particular character (with a specific B-ranking) in the **F** column ...
- Reconstructing the full **F** column would be easy: simply sort the **L** column lexicographically



# Reversing the BWT

- We will reconstruct the original string *T* backwards, from **right to left**
- We know the last character is \$, so we initialize our reconstructed string accordingly

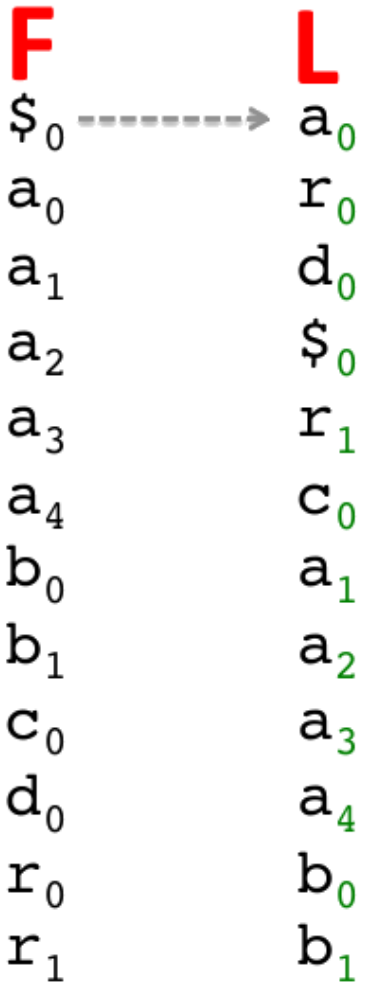
| F | L |
|---|---|
| $$_0$$ | $$a_0$$ |
| $$a_0$$ | $$r_0$$ |
| $$a_1$$ | $$d_0$$ |
| $$a_2$$ | $$\$_0$$ |
| $$a_3$$ | $$r_1$$ |
| $$a_4$$ | $$c_0$$ |
| $$b_0$$ | $$a_1$$ |
| $$b_0$$ | $$a_2$$ |
| $$c_0$$ | $$a_3$$ |
| $$d_0$$ | $$a_4$$ |
| $$r_0$$ | $$b_0$$ |
| $$r_1$$ | $$b_1$$ |

Reconstruction to date

T=... \$

# Reversing the BWT

- The character that precedes the \$ in *T* is now in the last column (**L**): Because of the **rotation** we know that the character preceding the \$ must be an *a* (here: $a_0$)

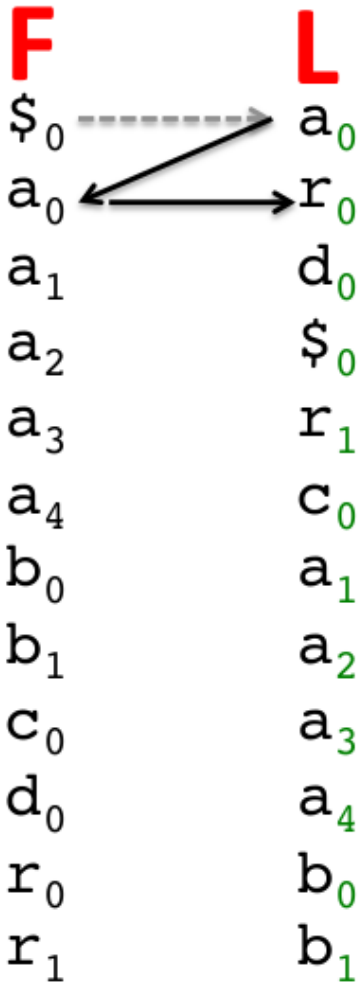



Reconstruction to date

T= ... $a_0$\$

# Reversing the BWT

- Because of the **cumulative index property** and because *a* comes right after $, we go to the second row of the BWM and find the same $a_0$
- The character that precedes the $a_0$ in T is again in the last (**L**) column (rotation!): $r_0$



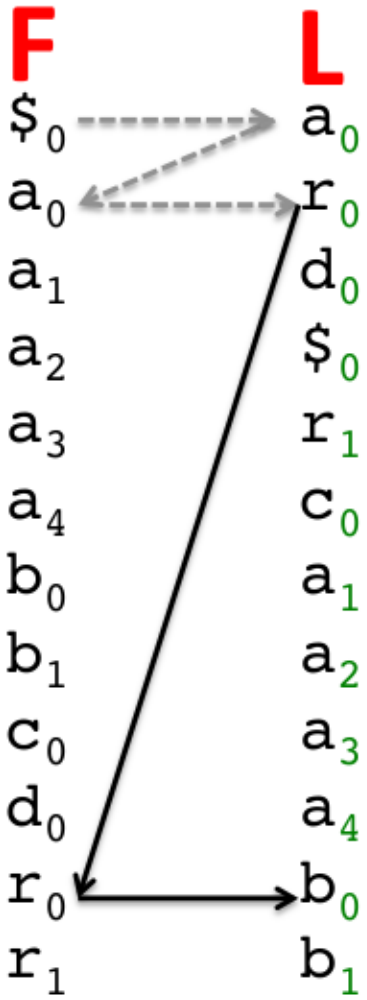

Reconstruction to date

T= ... $r_0 a_0 \$$

# Reversing the BWT

- To find the position of $r_0$ in the first column, we note that its index must be 1+5+2+1+1+0=10 because of the **cumulative index property**!
  (There are one "$", five "*a*", two "*b*", one "*c*", one "*d*", and zero "*r*" before $r_0$)

- We go to column **L** to get the next preceding character

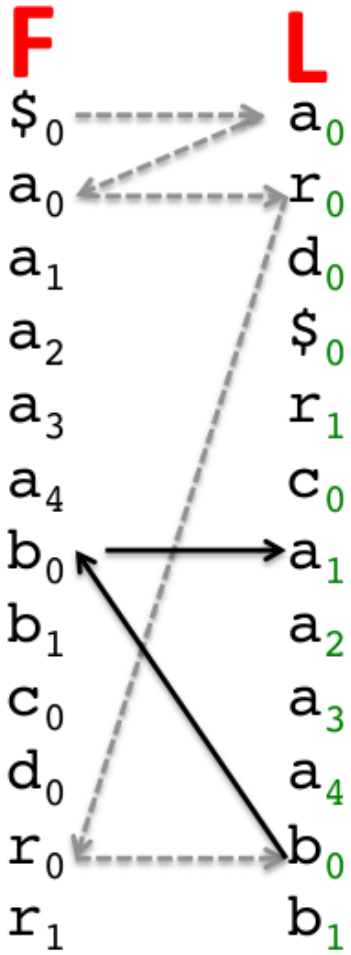



Reconstruction to date

T= ... $b_0 r_0 a_0 \$$

# Reversing the BWT

- The game continues...
- To find the position of $b_0$ in the first column, we note that its index must be 1+5+0=6 because of the **cumulative index property**
- We go to column **L** to get the next preceding character



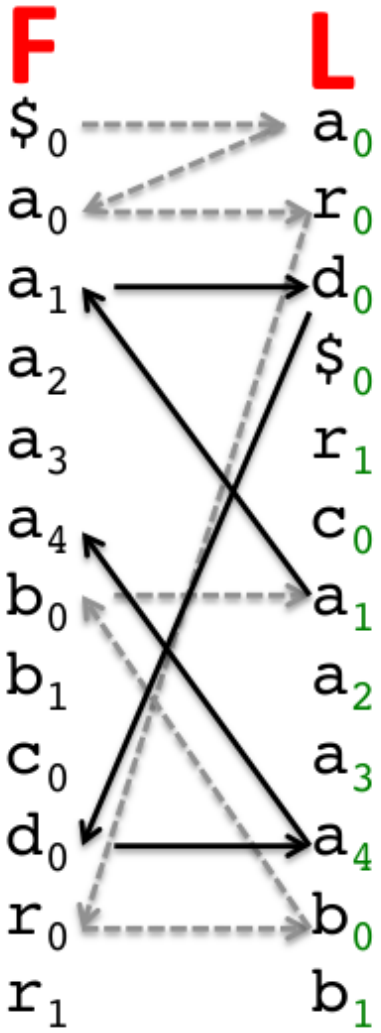

Reconstruction to date

T= ... $a_1 b_0 r_0 a_0 \$$

# Reversing the BWT

- Note that to find the position of $a_4$ with the **cumulative index property**, we take into account the preceding characters (here only \$), as well as the preceding indices of the same character (here: $a_0, a_1, a_2, a_3$), so that the index of $a_4$ in **F** is 1+4=5! (One "\$" and four other "*a*"s are preceding $a_4$)

- and so on...




Reconstruction to date

T= ... $a_4 d_0 a_1 b_0 r_0 a_0 \$$

# Reversing the BWT

But what information exactly did we need to do this reversal?

- We can do everything starting **only** from the BWT($T$), which corresponds to the **L** column!
- If we count the number of each character in BWT($T$), we can easily reconstruct the "chunks" of characters in the first (**F**) column of the BWM
  - We don't need to fully reconstruct the **F** column, we just need to know how many characters of each type it contains![1]
- We don't need to reconstruct the entire BW matrix!

[1] We can store this information in an array of size $\mathcal{O}(|\Sigma|)$ for characters in some alphabet $\Sigma$

# Alternative definition of the BWT

Please note, sometimes you will see a different way of defining the BWT:

- Instead of using a termination character ($), you can use the plain input string, compute the BWT (now without $) and store an *additional integer* which indicates at what position in the BWT one can find the *last* character of the original string
  - Without $, this integer is needed to know from where to reconstruct the original string
  - Advantage of using $: Since $ is by definition the first character in lexicographical order, the start point for reconstruction is always the first row of the BWM (that's why we don't need to store information about where to start the reconstruction!)
  - Disadvantage of using $: the BWM has one additional line (one more character = one more rotation), and we need to be *sure* that the termination character (e.g., $) cannot be part of the input string $T$ itself!

| | | |
|---|---|---|
| $a_0b_0r_0a_1c_0a_2d_0a_3b_1r_1a_4$ | $a_4a_0b_0r_0a_1c_0a_2d_0a_3b_1r_1$ | [0] |
| $b_0r_0a_1c_0a_2d_0a_3b_1r_1a_4a_0$ | $a_3b_1r_1a_4a_0b_0r_0a_1c_0a_2d_0$ | [1] |
| $r_0a_1c_0a_2d_0a_3b_1r_1a_4a_0b_0$ | $a_0b_0r_0a_1c_0a_2d_0a_3b_1r_1a_4$ | [2] |
| $a_1c_0a_2d_0a_3b_1r_1a_4a_0b_0r_0$ | $a_1c_0a_2d_0a_3b_1r_1a_4a_0b_0r_0$ | [3] |
| $c_0a_2d_0a_3b_1r_1a_4a_0b_0r_0a_1$ | $a_2d_0a_3b_1r_1a_4a_0b_0r_0a_1c_0$ | [4] |
| $a_2d_0a_3b_1r_1a_4a_0b_0r_0a_1c_0$ | $b_1r_1a_4a_0b_0r_0a_1c_0a_2d_0a_3$ | [5] |
| $d_0a_3b_1r_1a_4a_0b_0r_0a_1c_0a_2$ | $b_0r_0a_1c_0a_2d_0a_3b_1r_1a_4a_0$ | [6] |
| $a_3b_1r_1a_4a_0b_0r_0a_1c_0a_2d_0$ | $c_0a_2d_0a_3b_1r_1a_4a_0b_0r_0a_1$ | [7] |
| $b_1r_1a_4a_0b_0r_0a_1c_0a_2d_0a_3$ | $d_0a_3b_1r_1a_4a_0b_0r_0a_1c_0a_2$ | [8] |
| $r_1a_4a_0b_0r_0a_1c_0a_2d_0a_3b_1$ | $r_1a_4a_0b_0r_0a_1c_0a_2d_0a_3b_1$ | [9] |
| $a_4a_0b_0r_0a_1c_0a_2d_0a_3b_1r_1$ | $r_0a_1c_0a_2d_0a_3b_1r_1a_4a_0b_0$ | [0] |

Example (with $T$-ranking shown):
$T$ = abracadabra

BWT($T$) = rdarcaaaabb
last_char_pos = 2

Note:
With $ the BWT was:
BWT($T$) = ard$rcaaaabb (!)

**We will use the $ !**

# FM Index

The FM index uses the BWT and some other auxilliary data structures to generate a fast an efficient index for search for small patterns within a larger string $T$

- "FM" stands for "**F**ull-text index in **M**inute space"
  (or for "**F**erragina–**M**anzini")
- Paolo Ferragina and Giovanni Manzini (2000): "Opportunistic Data Structures with Applications". *Proceedings of the 41st Annual Symposium on Foundations of Computer Science*, pp. 390–398

# FM Index

- The main data structures of the FM index are the **F** column and the **L** column (=BWT) from the BWM
  - Note that the **F** column itself is not stored, because it can be represented as an array of integers (one per character of our alphabet)
  - In our example, and using the order

    $$\$ < a < b < c < d < r$$

    we have

| 1 | 5 | 2 | 1 | 1 | 2 |
|---|---|---|---|---|---|

  - As mentioned, the block-sorted **L** column is also easily compressible
- We will add auxiliary data structures to this ...

| **F** | **L** |
|---|---|
| $_0$ | abracadabra$_0$ |
| a$_0$ | $abracadabr$_0$ |
| a$_1$ | bra$abracad$_0$ |
| a$_2$ | bracadabra$$_0$ |
| a$_3$ | cadabra$abr$_1$ |
| a$_4$ | dabra$abrac$_0$ |
| b$_0$ | ra$abracada$_1$ |
| b$_1$ | racadabra$a$_2$ |
| c$_0$ | adabra$abra$_3$ |
| d$_0$ | abra$abraca$_4$ |
| r$_0$ | a$abracadab$_0$ |
| r$_1$ | acadabra$ab$_1$ |

# FM Index

- But how can we search for a particular substring (e.g., for a read)?
  - Is it present?
  - If so, at what position(s)?

- As mentioned, the BWM is very similar to a suffix array:
  - Same sorting
  - Strings in the BWM until $ are the suffixes

  The substring we're looking for should be the prefix of a suffix in the BWM, but a binary search over just **F** and **L** is obviously not possible (the "middle" of the matrix is missing)

- We will again make use of the B-ranks and take an approach which is somewhat similar to the reconstruction of the original string
  - We will try to reconstruct the prefixes of suffixes which fit the searched substring!

| F | L |
|---|---|
| $$_0$$ | abracadabra$_0$ |
| a$_0$ | $abracadabr$_0$ |
| a$_1$ | bra$abracad$_0$ |
| a$_2$ | bracadabra$$_0$ |
| a$_3$ | cadabra$abr$_1$ |
| a$_4$ | dabra$abrac$_0$ |
| b$_0$ | ra$abracada$_1$ |
| b$_1$ | racadabra$a$_2$ |
| c$_0$ | adabra$abra$_3$ |
| d$_0$ | abra$abraca$_4$ |
| r$_0$ | a$abracadab$_0$ |
| r$_1$ | acadabra$ab$_1$ |

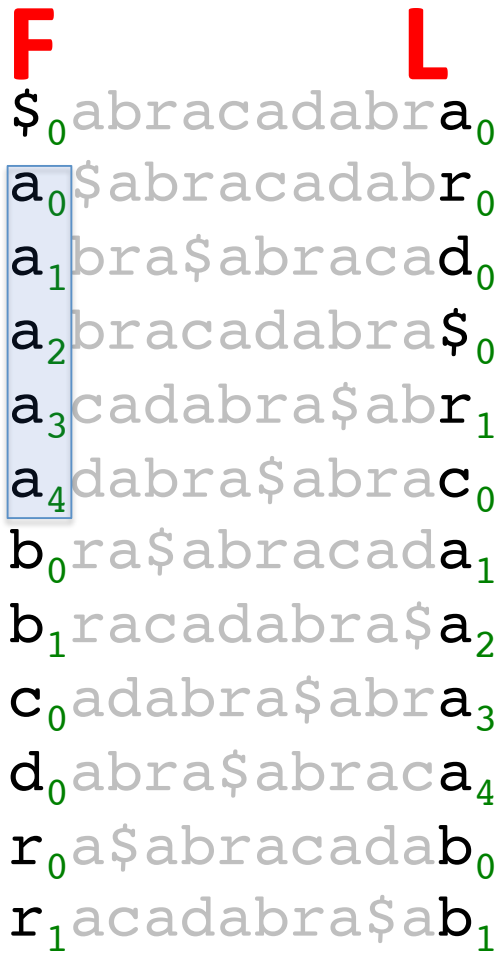
# FM Index

- For example, let us search for the string `P=abra` in our "genome" `T=abracadabra`
- Our strategy is to look for all rows of BWM(T) that have P as a prefix, **without** reconstructing the entire BWM!
- We successively look for the longer suffixes which have prefix P, **starting with the last** character of P (here: an "a")
- But it is easy to find the chunk of the BWM(T) that starts with a given character using the **cumulative index property**:

$ < **a** < b < c < d < r

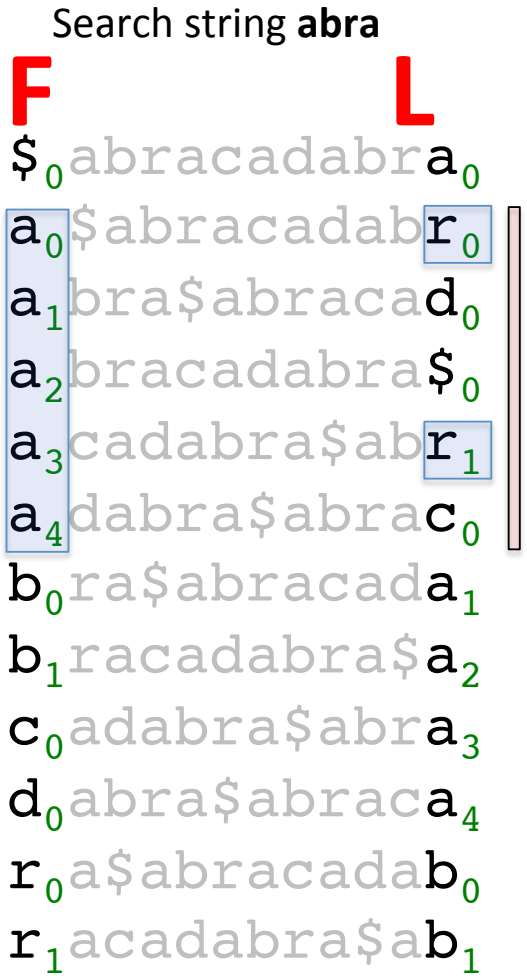
| 1 | **5** | 2 | 1 | 1 | 2 |
|---|---|---|---|---|---|

Search string **abra**



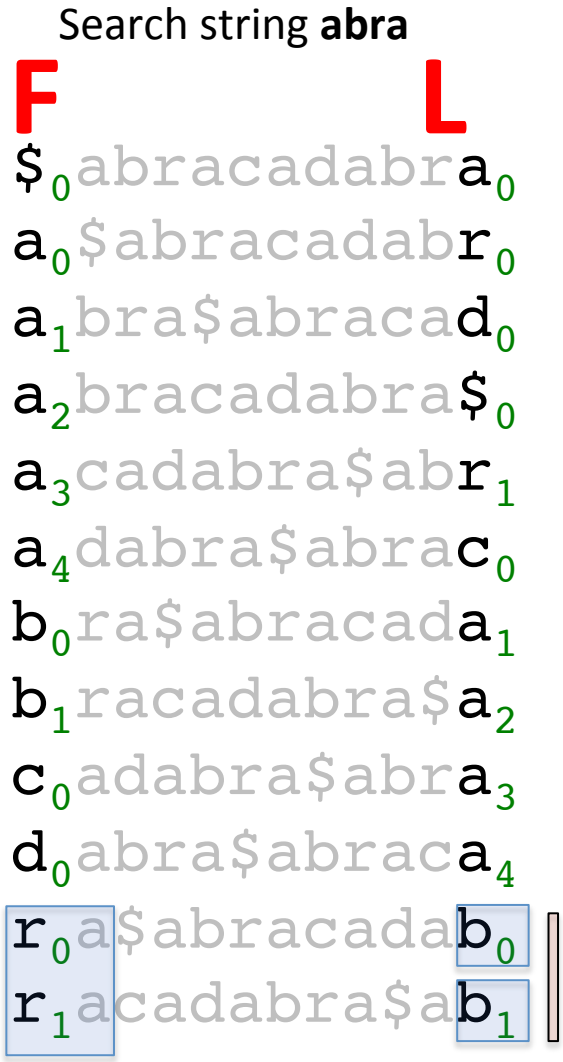
# FM Index

- Once we have found all rows that begin with the last letter of P, we can look in **L** to identify those rows whose last letter corresponds to the second to last letter in P (here: letter *r* of *abra*)
  - Due to rotation, the last letter in the row (in column **L**) must come right before the first letter (column **F**) in the original string!
- We can now read off the B-ranks of these characters (here: 0 and 1) and use the LF mapping to find the rows in **F** that begin with these characters
  - Note: we are guaranteed to have consecutive B-ranks for the second letter (here: r) because we're looking at a set of adjacent rows!



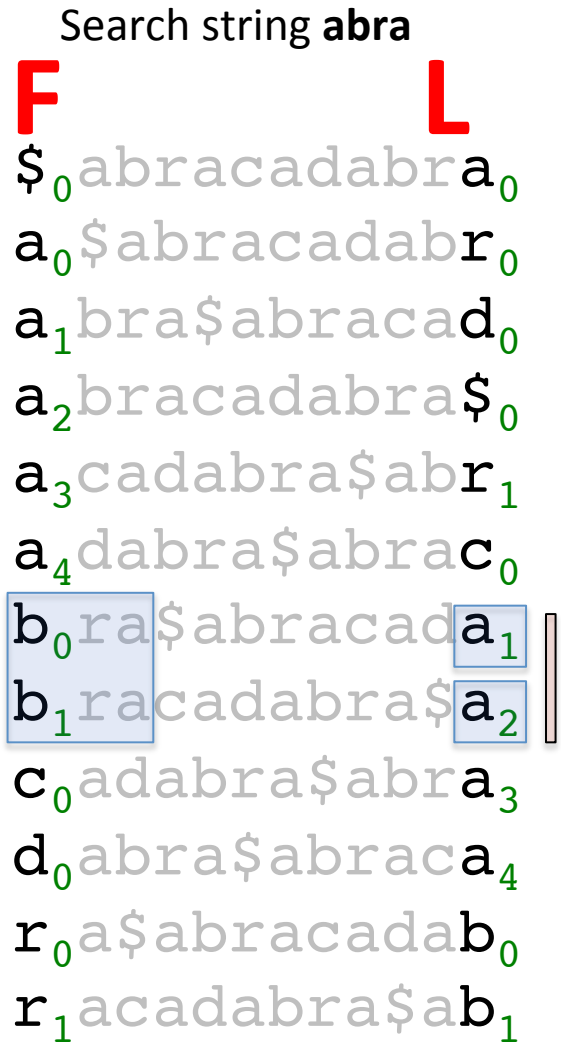
# FM Index

- Using the LF mapping we find the rows in **F** that begin with "*ra*" ($r_0$ and $r_1$)
- The character that precedes "*r*" in our query string P="abra" is "*b*", so we can continue
  - Again, we check the rows which start with $r_0$ and $r_1$ for occurrences of "*b*" in the last column!
  - In this case: both of them have a "*b*", which before the rotation was right in front of the "*ra*" at the beginning of the rows
    - ★ We take note of their B-ranks: From 0 to 1 ($b_0$ and $b_1$)
- We have now matched the last 3 characters of `P=abra` and continue one more step using the LF mapping



# FM Index

- We find the rows that begin with "*bra*" ($b_0$ and $b_1$) and look at the corresponding characters in **L** to see if we have a match for P
  - Check **L** for the first character in P (the only remaining character we need to check)
  - Both rows have an "*a*" (B-ranks 1 to 2), so both match P!



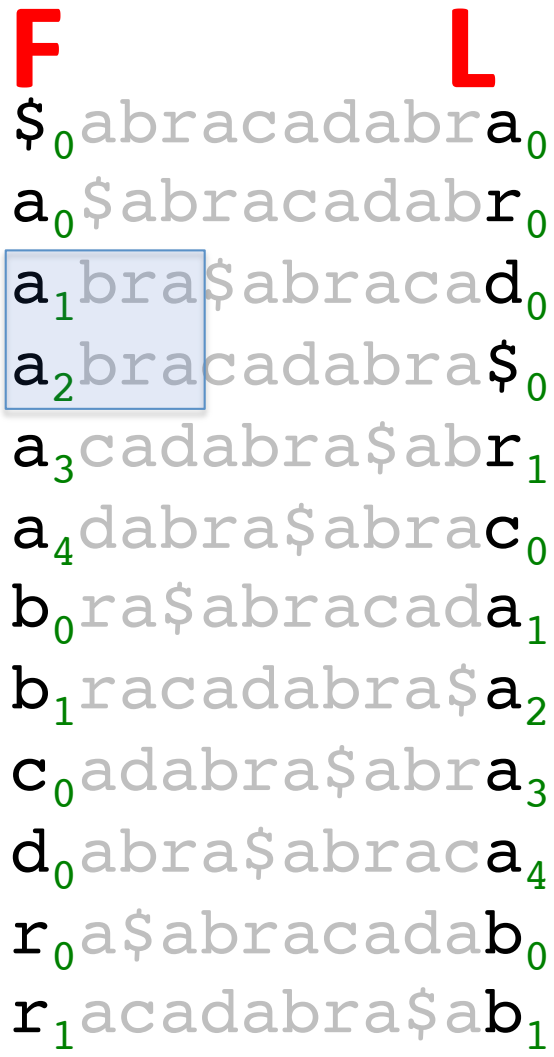
# FM Index

- Finally, we look up $a_1$ and $a_2$ in the **F** column to find the rows of the BWM that begin with our full query string: $[2, 4)$
  - Remember: we are guaranteed to have consecutive B-ranks, so we will by definition have consecutive rows!
  - Actually, in each of the steps when we looked up letters in **F** we had consecutive rows ...
  - We didn't need to reconstruct the matrix; all necessary information is in **F** and **L**=BWT!

- These are equivalent to the rows we would have identified with a binary search over the suffix array (which is of course an array of start positions of suffixes)

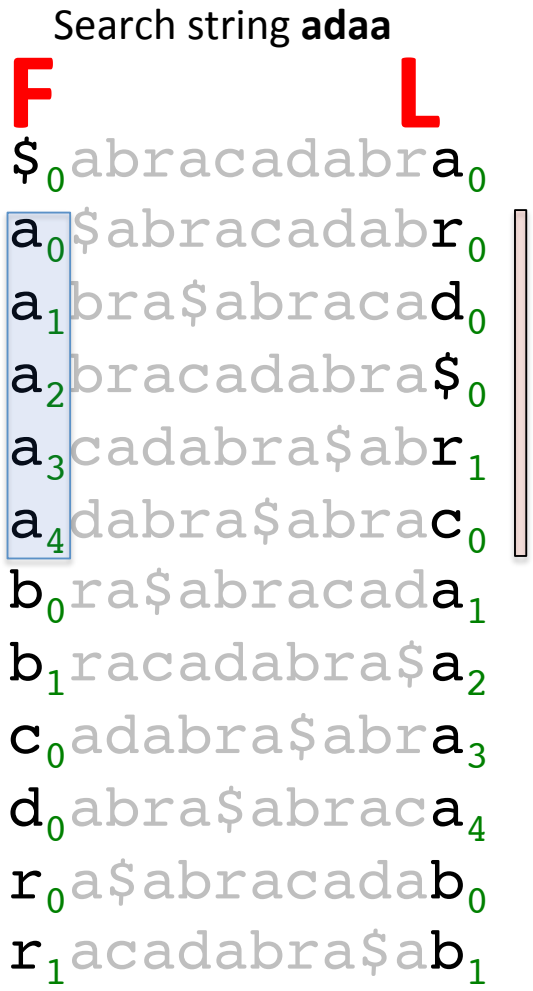
- However, it is not immediately clear how to identify the positions in `T` that correspond to `P` when using the BWT (and F) alone

Search string **abra**



# FM Index

- What about the search pattern `P=adaa`?
- We start matching the last character in the **F** column as previously
- But: when we now look at the letters in the **L** column of the corresponding rows, there is no "a" (second last letter in P)
  - Thus, there is no "*aa*" in the original string!
  - Ergo, the full search pattern *adaa* does not occur in `T`!
- Once we have a mismatch, we end the search
  - At least for now, we will see how to cope with mismatches later ...
  - Since we want to identify mutations, we ultimately need to be able to map reads also if they do not precisely match the reference genome; for now, we ignore that problem



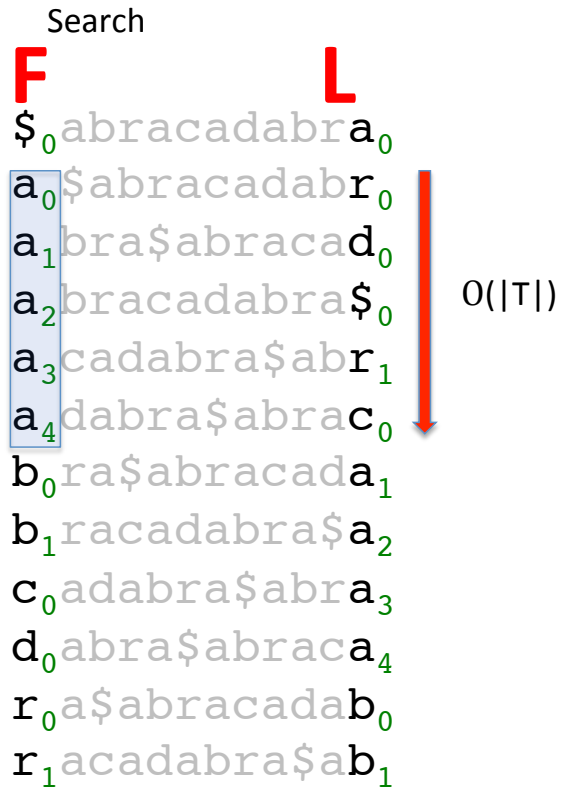
# FM Index – Interim Report

We have presented a somewhat naive version of the FM index search. However, we have glossed over three issues that need to be solved to produce an efficient and practical algorithm

## Issue #1

- How do we efficiently find the preceding character (i.e., starting from a chunk of prefixes in or starting in **F**, how do we find the correct characters in **L** to continue leftwards)?

- In the worst case, we may have to scan down as far as the length of the entire input string, $\mathcal{O}(|T|)$
  - $|T| > 3$ billion for the human genome!



# FM Index – Interim Report

**Issue #2**

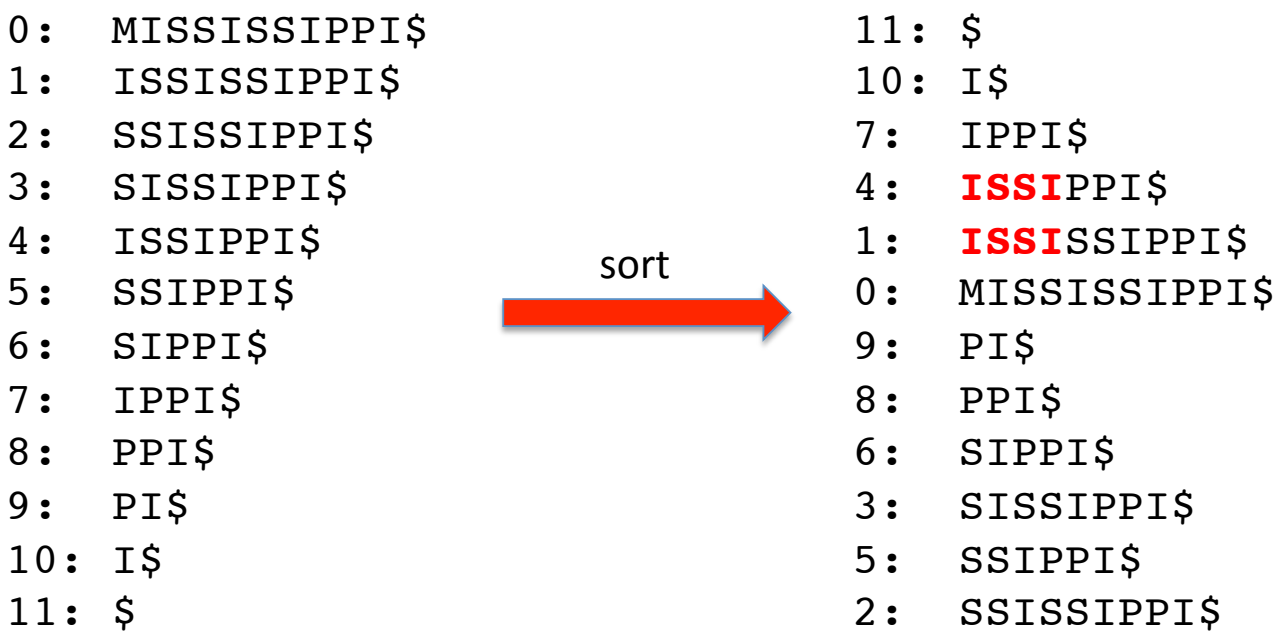
- Recall that we did not want to **explicity** store the B-ranks of the characters – this would be at least 4 bytes per input character, and whatever advantage we had with respect to the suffix array would disappear
- So, we still need a way of getting the B-ranks of the characters in **L**

| F | L |
|---|---|
| $$_0$$abracadabr | a$_0$ |
| a$_0$$abracadabr | r$_0$ |
| a$_1$bra$abracac | d$_0$ |
| a$_2$bracadabra$ | $$_0$ |
| a$_3$cadabra$abr | r$_1$ |
| a$_4$dabra$abrac | c$_0$ |
| b$_0$ra$abracada | a$_1$ |
| b$_1$racadabra$a | a$_2$ |
| c$_0$adabra$abra | a$_3$ |
| d$_0$abra$abraca | a$_4$ |
| r$_0$a$abracadab | b$_0$ |
| r$_1$acadabra$ab | b$_1$ |

# FM Index – Interim Report

## Issue #3

- Recall that in the suffix array, we explicitly stored the start position of each suffix of $T$
- We do not have this information with the BWM
- So, we still need a way of figuring out at what positions matches occur in $T$



# FM Index – Tally Table

Issue #1: efficiently find the preceding character(s) in **L**

- Construct a tally table: ("tally" $\approx$ counter)
  - Precalculate the number of each specific character in **L** up to every row
  - (In general, we can substract one from the tally to obtain the **zero-based B-rank**; i.e., *B-rank* $=$ *tally* $-1$)

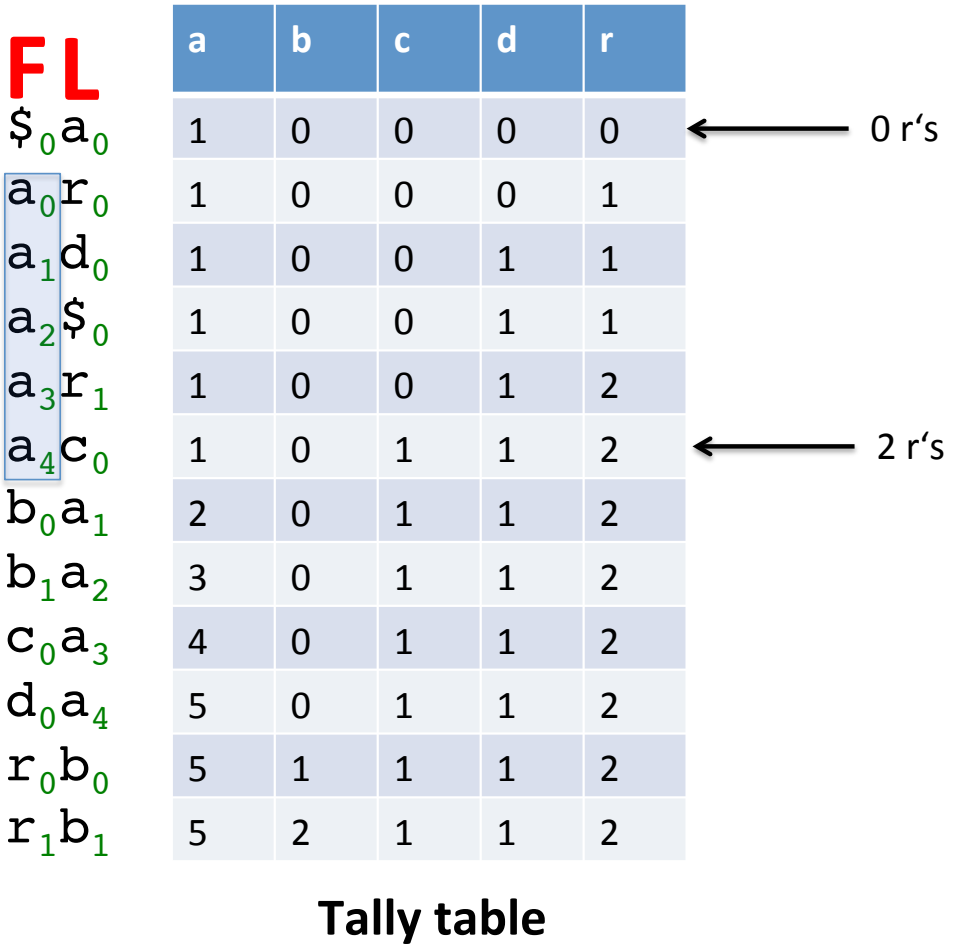
| **F** | **L** |
|---|---|
| $_0a_0$ | $a_0r_0$ |
| $_1a_0$ | $a_1d_0$ |
| $_2a_0$ | $a_2\$_0$ |
| $_3a_1$ | $a_3r_1$ |
| $_4a_1$ | $a_4c_0$ |
| $_0b_1$ | $b_0a_1$ |
| $_1b_2$ | $b_1a_2$ |
| $_0c_3$ | $c_0a_3$ |
| $_0d_4$ | $d_0a_4$ |
| $_0r_5$ | $r_0b_0$ |
| $_1r_5$ | $r_1b_1$ |

| a | b | c | d | r |
|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 0 | 1 | 2 |
| 1 | 0 | 1 | 1 | 2 |
| 2 | 0 | 1 | 1 | 2 |
| 3 | 0 | 1 | 1 | 2 |
| 4 | 0 | 1 | 1 | 2 |
| 5 | 0 | 1 | 1 | 2 |
| 5 | 1 | 1 | 1 | 2 |
| 5 | 2 | 1 | 1 | 2 |

**Tally table**

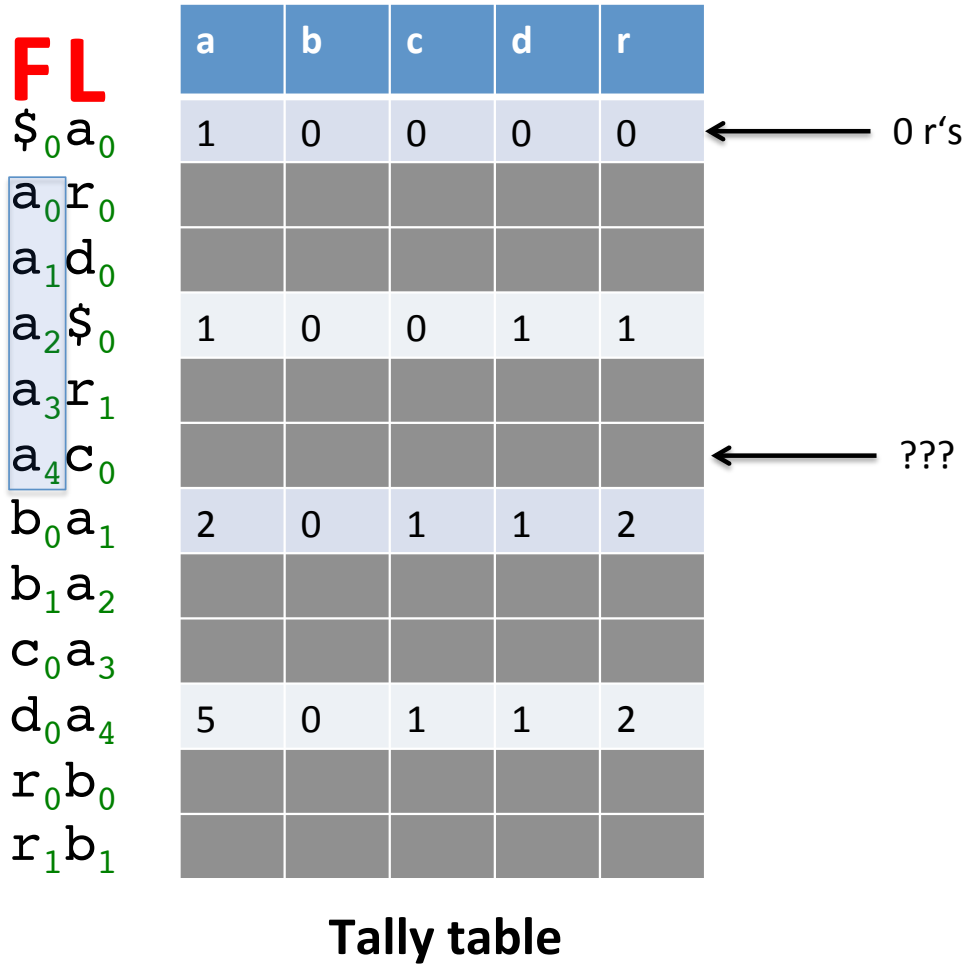
# FM Index – Tally Table

- Say we are search for `P=abra`
- After we have found all rows beginning with *a* in the first step, we need to find rows with *r* in the **L** column
- Say the range of rows is $[i, j]$
- We look in the tally table in row $i-1$. No occurences of *r* before *i*!
- Now look in the tally table row *j*. Two occurences of *r* at *j*!
- Therefore, we know that (only) $r_0$ and $r_1$ occur in **L** in the range $[i, j]$
  Two lookups instead of $\mathcal{O}(|T|)$ !



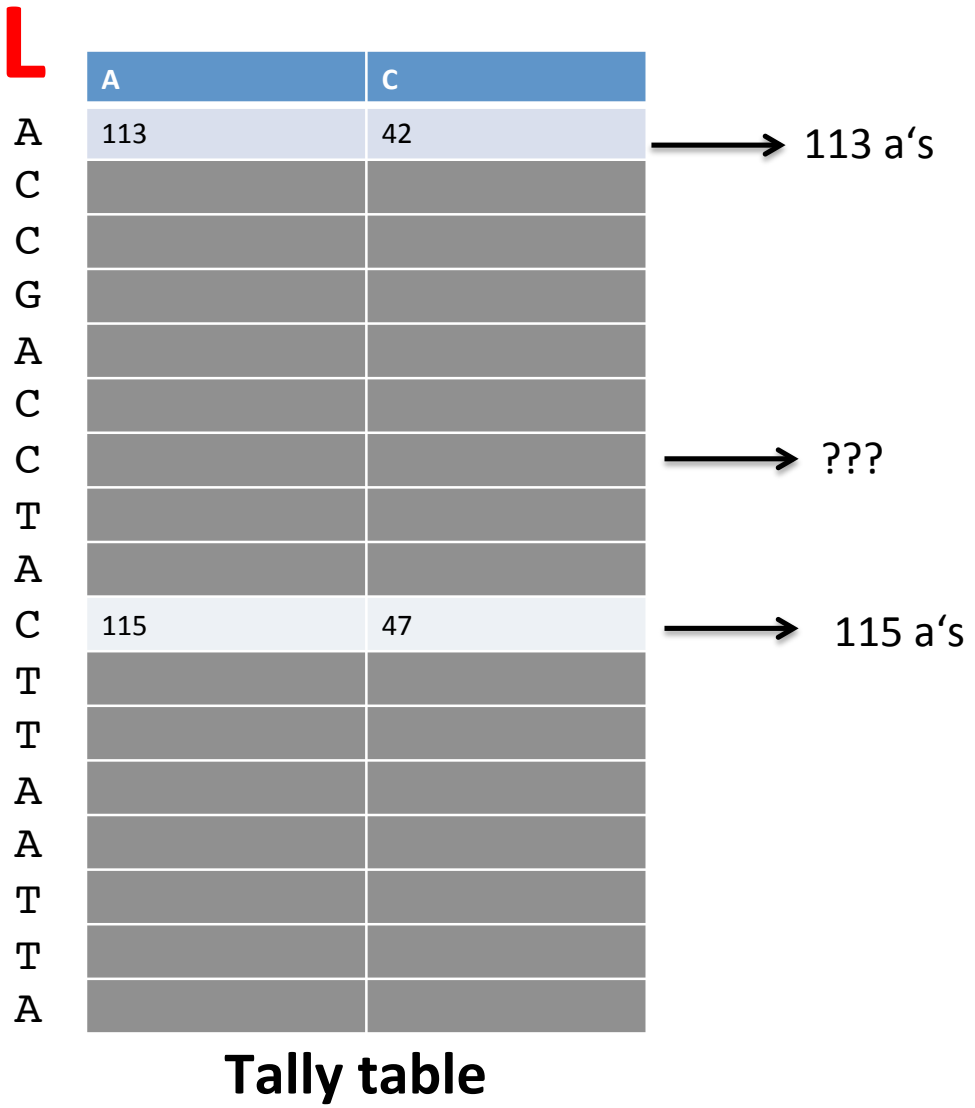
# FM Index – Tally Table

- A problem with this idea is that we need to store $\mathcal{O}(|T| \cdot |\Sigma|)$ integers
- What if we store only every $k^{\text{th}}$ row?
  - "checkpoints"
- We reduce the size of the tally table by a factor of $k$, but at the price of not having all of the information we need immediately available



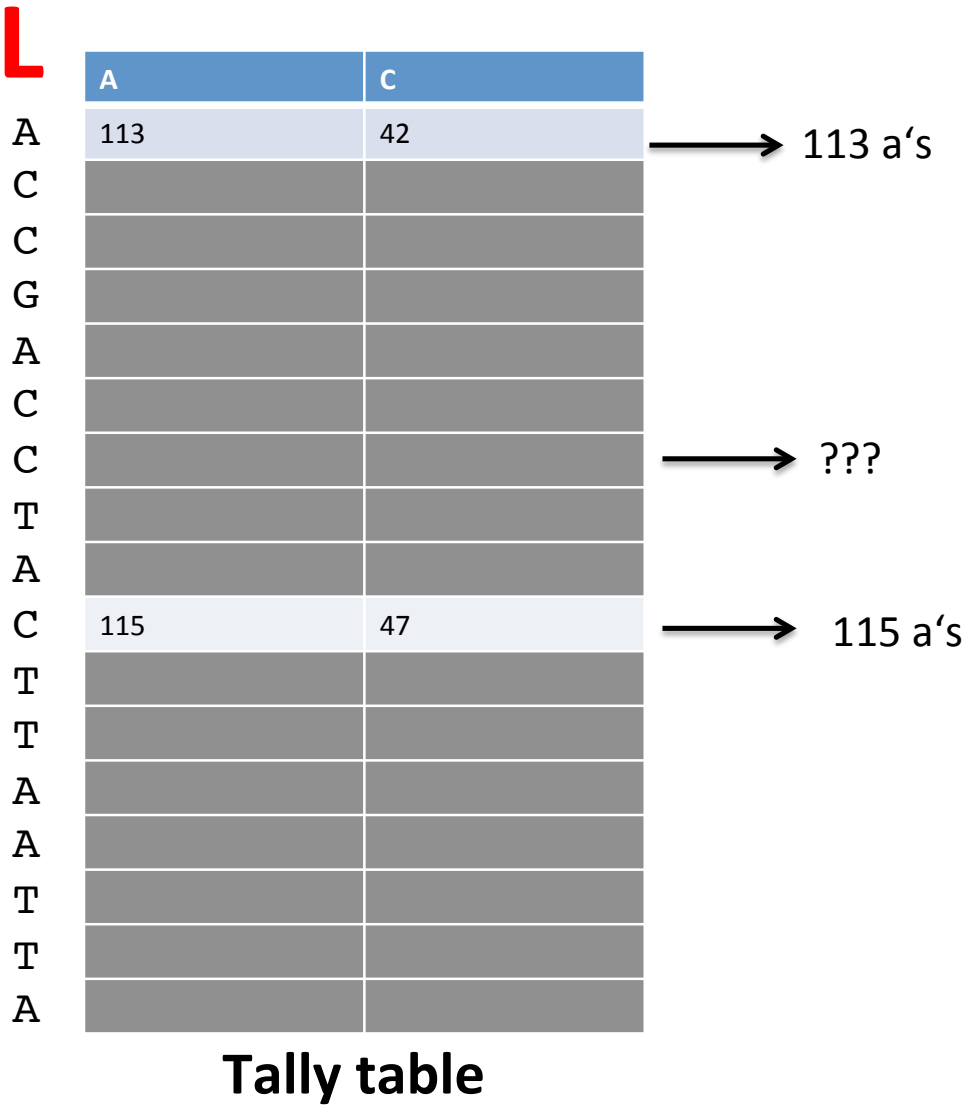
# FM Index – Tally Table

- For instance, to calculate the rank of the A near the ← ???
- We can go to the previous checkpoint and count the number of A's that we encounter from there to the position we are interested in: $113 + 1 = 114$
- Or: We can go to the next checkpoint and substract the number of A's that we encounter along the way: $115 - 1 = 114$



# FM Index – Tally Table

- Assuming we space the check point rows a constant number of rows away from one another ($k$ is constant): $\mathcal{O}(1)$, for instance, 50 rows, then lookups are still $\mathcal{O}(1)$ rather than $\mathcal{O}(|T|)$
- We now also have a way of getting the B-ranks we need for issue # 2
- In general, we will substract one from the tally to obtain the **zero-based B-rank**
- (The storage space needed is still $\mathcal{O}(|T|)$, but with a smaller factor)



# FM Index – Finding indices in T

| F | L | | |
|---|---|---|---|
| $$_0$$abracadabr | a$$_0$$ | [11] | $ |
| a$$_0$$$abracadab | r$$_0$$ | [10] | a$ |
| a$$_1$$bra$abraca | d$$_0$$ | [7] | abra$ |
| a$$_2$$bracadabra | $$_0$$ | [0] | abracadabra$ |
| a$$_3$$cadabra$ab | r$$_1$$ | [3] | acadabra$ |
| a$$_4$$dabra$abra | c$$_0$$ | [5] | adabra$ |
| b$$_0$$ra$abracad | a$$_1$$ | [8] | bra$ |
| b$$_1$$racadabra$ | a$$_2$$ | [1] | bracadabra$ |
| c$$_0$$adabra$abr | a$$_3$$ | [4] | cadabra$ |
| d$$_0$$abra$abrac | a$$_4$$ | [6] | dabra$ |
| r$$_0$$a$abracada | b$$_0$$ | [9] | ra$ |
| r$$_1$$acadabra$a | b$$_1$$ | [2] | racadabra$ |

- Issue #3 referred to the desire to have information, like in the suffix array, that would allow us to find the position of matches in the original string
- Recall the suffixes to which the suffix array refers (which stores the indices/positions in $T$, not the suffixes themselves) are equivalent to the strings of the BWM (until the $)

# FM Index – Finding indices in T

| F | L | | |
|---|---|---|---|
| $$_0$$abracadabra | a$_0$ | [11] | $ |
| a$_0$$abracadabr | r$_0$ | [10] | a$ |
| a$_1$bra$abracad | d$_0$ | [7] | abra$ |
| a$_2$bracadabra$ | $$_0$$ | [0] | abracadabra$ |
| a$_3$cadabra$abr | r$_1$ | [3] | acadabra$ |
| a$_4$dabra$abrac | c$_0$ | [5] | adabra$ |
| b$_0$ra$abracada | a$_1$ | [8] | bra$ |
| b$_1$racadabra$a | a$_2$ | [1] | bracadabra$ |
| c$_0$adabra$abra | a$_3$ | [4] | cadabra$ |
| d$_0$abra$abraca | a$_4$ | [6] | dabra$ |
| r$_0$a$abracada | b$_0$ | [9] | ra$ |
| r$_1$acadabra$a | b$_1$ | [2] | racadabra$ |

**abra**cad**abra**$
**abra** Pos. 0
**abra** Pos. 7

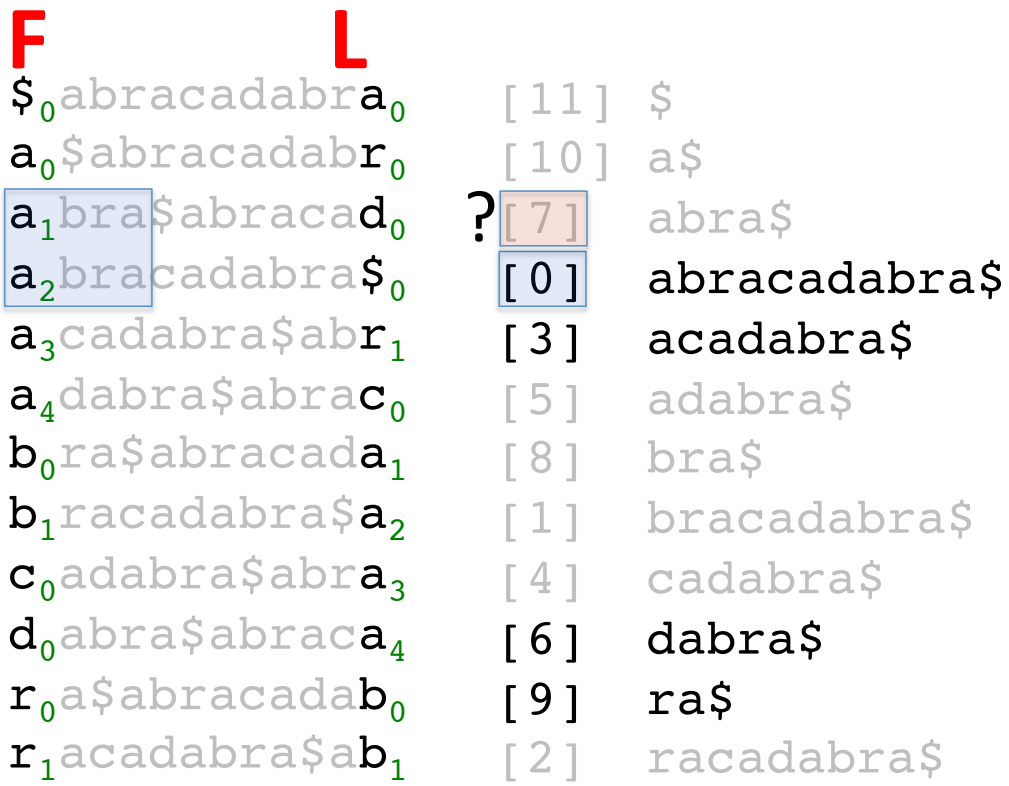
- For instance, if we had just used the algorithm described above to find two occurences of the pattern `abra`, then we could look up their start positions in T (here: 0 and 7) if we also had the suffix array!

# FM Index – Finding indices in T

| F | L | | |
|---|---|---|---|
| $$_0$ | abracadabra | a$_0$ | [11] $ |
| a$_0$ | $abracadabr | a$_0$ | [10] a$ |
| a$_1$ | bra$abracad | a$_0$ | [7] abra$ |
| a$_2$ | bracadabra$ | $$_0$ | [0] abracadabra$ |
| a$_3$ | cadabra$abr | a$_1$ | [3] acadabra$ |
| a$_4$ | dabra$abrac | a$_0$ | [5] adabra$ |
| b$_0$ | ra$abracad | a$_1$ | [8] bra$ |
| b$_1$ | racadabra$ | a$_2$ | [1] bracadabra$ |
| c$_0$ | adabra$abr | a$_3$ | [4] cadabra$ |
| d$_0$ | abra$abrac | a$_4$ | [6] dabra$ |
| r$_0$ | a$abracad | ab$_0$ | [9] ra$ |
| r$_1$ | acadabra$a | b$_1$ | [2] racadabra$ |

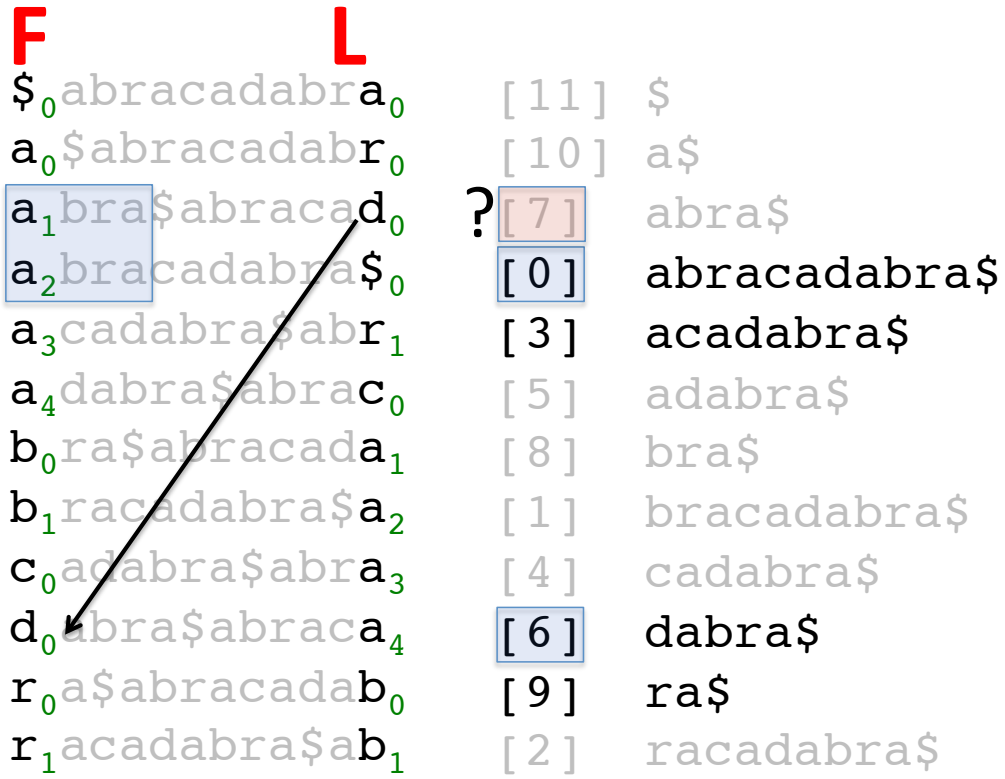
- But, if we stored the entire suffix array, this would incur roughly an additional $4 \times |\mathtt{T}|$ bytes of storage
- We can use the same checkpoint idea as for the tally table:
  - Don't store all of the values of the suffix array, just store every $k^{th}$ value
  - Importantly, we store every $k^{th}$ value for the original string $\mathtt{T}$ (e.g., indexes 0,3,6,9,... in $T$), not every $k^{th}$ value in the suffix array – this ensures constant time

# FM Index – Finding indices in T



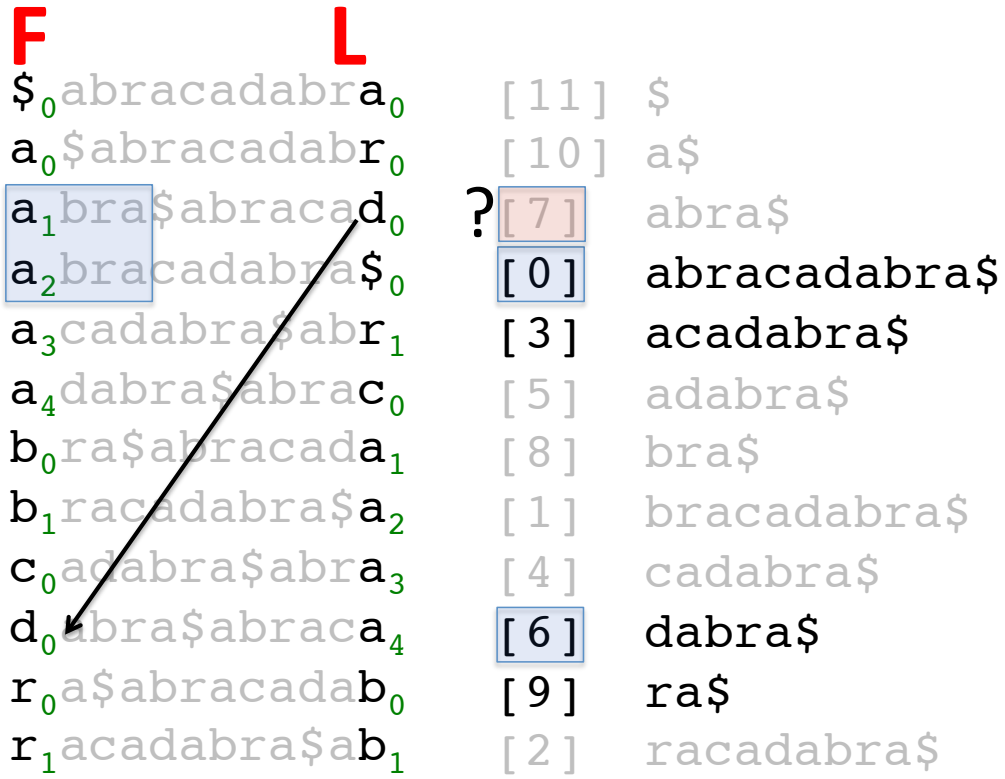

- So, let's again search for the pattern `P=abra`
- We find one hit and our "selective suffix array" indicates the match to occur at position 0 of the original string ("`abracadabra$`")
- What do we do about the other hit?

# FM Index – Finding indices in T




- To find the position of the match, we can take advantage of the LF mapping:
  - This tells us where to find the $d_0$ in the first column **F**
  - We can look this up in our selective suffix array – but note that we have moved one position to the left – the position of `dabra` is 6, so the position of `abra` must be 7!

# FM Index – Finding indices in T




- Note that the fact that we are storing every $k^{th}$ value for the original string $\mathtt{T}$, ensures that we need to perform at most $k-1$ "hops" to retrieve the index we are looking for
- However, we are still keeping $\mathcal{O}(|T|)$ elements in the selective suffix array

# FM Index – Memory footprint

The FM index of the human genome has a substantially smaller memory footprint than the suffix tree (at least 60 GB) or the suffix array (at least 12 GB)

| Component | Complexity | Size (for the human genome) |
|---|---|---|
| F | $\mathcal{O}(\lvert\Sigma\rvert)$ | 16 bytes (4 integers) |
| L (=BWT) | $\lvert T\rvert$ chars | 2 bits $\times 3.1 \times 10^9 \approx$ 750 MB $^{(i)}$ |
| selective SA | $\sim \frac{1}{k}\lvert T\rvert$ integers | $\approx$400 MB with $k = 32$ $^{(ii)}$ |
| tally table checkpoints | $\sim \frac{1}{x}\lvert T\rvert \cdot \lvert\Sigma\rvert$ integers | $\approx$400 MB with $x = 128$ $^{(ii)}$ |

$\rightarrow$ Total size for the FM index of the human genome thus about 1.5 GB

Notes:

- (i) We store the 4 nucleotides with 2 bits each, i.e., 4 nucleotides per byte
- (ii) $k$ and $x$ are the lengths of the skips; 4 bytes per integer

# References I




Burrows M. & Wheeler D.J. (1994)
*A block-sorting lossless data compression algorithm*
Technical report 124; Palo Alto, CA:*Digital Equipment Corporation*



Ferragina P. & Mazzini P. (2000)
*Opportunistic Data Structures with Applications*
Proceedings of the 41st IEEE Symposium on Foundations of Computer Science, pp. 390–398